

Bom dia! Já vamos começar

1. Sente no mesmo computador de segunda e abra a sua pasta no VS Code
2. Confira se o `jogador.json` está lá dentro
3. Faltou na segunda ou perdeu o arquivo? No repositório, baixe o `dados_offline/jogador_exemplo.json` e salve na sua pasta com o nome `jogador.json`
4. Computador diferente do de segunda? Rode o `verificar_ambiente.py`

Repositório: tinyurl.com/secompp-apis

Consumo de APIs e Análise de Dados na Prática

Felipe W. Giovaneli e Victor B. Zucherato

O plano de hoje

Horário	Quarta (30/09)
08:20	Revisão rápida da segunda
08:45	Análise de dados e DataFrame
09:10	Prática 3: pandas com os seus dados
10:00	Intervalo
10:30	Desafio livre: vocês escolhem a API
11:30	Apresentação dos resultados

Revisão



O que vimos na segunda

O que é API?

Imagine um
restaurante

- Cliente
- Cozinha
- Garçom



| Status Codes

200	OK	Sucesso! O servidor entendeu o pedido e enviou os dados esperados.
400	Bad Request	Erro do cliente. O formato da URL ou os parâmetros enviados contêm erros de digitação.
401	Unauthorized	Falta de identificação. Você não enviou o Token/Chave da API ou ele está ausente.
403	Forbidden	Acesso negado. Sua chave existe, mas você não tem permissão para acessar aquele recurso (ex: IP bloqueado).
404	Not Found	Não encontrado. O endpoint digitado não existe ou a TAG do jogador/item busca não foi localizada.
429	Too Many Requests	Você estourou o Rate Limit. Enviou requisições demais em um curto espaço de tempo.
500	Internal Server Error	Erro interno do servidor. A culpa não é sua; a API deles está instável ou fora do ar.

Navegando em dados aninhados

```
jogador = {  
    "name": "Victor",  
    "club": {"name": "Clube da FCT"},  
    "brawlers": [  
        {"name": "SHELLY", "trophies": 750},  
        {"name": "COLT", "trophies": 620},  
    ],  
}  
  
print(jogador["club"]["name"])      # Clube da FCT  
print(jogador["brawlers"][0]["name"]) # SHELLY  
  
for brawler in jogador["brawlers"]:  
    print(brawler["name"], brawler["trophies"])
```

O código de segunda, inteiro

```
import json
import requests

TOKEN = "COLE_A_CHAVE_AQUI"
cabecalhos = {"Authorization": f"Bearer {TOKEN}"}
url = "https://api.brawlstars.com/v1/players/%23RGVU2J"
resposta = requests.get(url, headers=cabecalhos)

if resposta.status_code == 200:
    jogador = resposta.json()
    with open("jogador.json", "w", encoding="utf-8") as arquivo:
        json.dump(jogador, arquivo, ensure_ascii=False, indent=2)
else:
    print("Erro:", resposta.status_code)
```

Teoria 3

Análise de dados e DataFrame

Analisar dados é responder perguntas

Dado bruto sozinho não diz nada. Tudo começa com uma pergunta:

- Qual é o meu brawler com mais troféus?
- Quantos brawlers eu tenho acima de 1100 troféus?
- Qual brawler está mais longe do próprio recorde?
- Em qual modo de jogo eu mais ganho?

O ciclo da análise

Coletar	Organizar	Perguntar	Responder
requests e a API	pandas monta a tabela	filtros, contagens e grupos	números, tabelas e gráficos

O JSON é bom para guardar...

...mas, para saber qual brawler tem mais troféus, teríamos que percorrer a lista com um **for**:

```
{
  "name": "Vovoito",
  "brawlers": [
    {"name": "SHELLY", "power": 11, "trophies": 1008},
    {"name": "COLT", "power": 11, "trophies": 1058},
    {"name": "BULL", "power": 11, "trophies": 1028},
    ...
  ]
}
```

...a tabela é boa para perguntar

name	power	trophies
SHELLY	11	1008
COLT	11	1058
BULL	11	1028
...	...	...

Cada dicionário da lista vira uma **linha** e cada chave vira uma **coluna**: isso é um **DataFrame**

Primeiro olhar numa tabela

Comando	O que faz
<code>pd.DataFrame(lista)</code>	Transforma a lista de dicionários em tabela
<code>df.head()</code>	Mostra as 5 primeiras linhas
<code>df.shape</code>	Diz quantas linhas e colunas a tabela tem
<code>df.info()</code>	Lista as colunas e o tipo de cada uma
<code>df.describe()</code>	Média, mínimo, máximo e outras estatísticas

Fazendo perguntas para a tabela

Comando	O que faz
<code>df[["a", "b"]]</code>	Escolhe colunas
<code>df[df["a"] > 10]</code>	Filtra linhas
<code>df.sort_values("a")</code>	Ordena
<code>df["a"].value_counts()</code>	Conta quantas vezes cada valor aparece
<code>df.groupby("a")["b"].mean()</code>	Média de b para cada valor de a
<code>df.to_csv("dados.csv")</code>	Salva para abrir no Excel

Prática 3



pandas com os seus dados

Do JSON para a tabela

Crie o arquivo **03_pandas.py** na mesma pasta do **jogador.json**

```
import json
import pandas as pd

with open("jogador.json", encoding="utf-8") as arquivo:
    jogador = json.load(arquivo)

df = pd.DataFrame(jogador["brawlers"])
print(df.head())
```

Primeiro olhar

Continuem no mesmo arquivo, embaixo do que já existe:

```
print(df.shape)  # (linhas, colunas)
df.info()
print(df["trophies"].describe())
```

Escolhendo colunas e ordenando

```
colunas = ["name", "power", "trophies", "highestTrophies"]  
tabela = df[colunas]  
  
top10 = tabela.sort_values("trophies", ascending=False).head(10)  
print(top10)
```

Filtrando linhas

Só ficam as linhas em que a condição nos colchetes é verdadeira:

```
fortes = tabela[tabela["trophies"] >= 1100]  
print(len(fortes), "brawlers com 1100 troféus ou mais")  
print(fortes)
```

Coluna nova

Dá para criar colunas fazendo conta com as outras, sem **for**:

```
df["perdidos"] = df["highestTrophies"] - df["trophies"]  
  
longe = df.sort_values("perdidos", ascending=False)  
print(longe[["name", "trophies", "perdidos"]].head())
```

Contando e agrupando

`value_counts` conta cada valor e `groupby` resume cada grupo:

```
print(df["power"].value_counts())
```

```
media = df.groupby("power")["trophies"].mean()  
print(media)
```

Salvando para abrir no Excel

Depois, abram o `brawlers.csv` no Excel ou no LibreOffice:

```
tabela.to_csv("brawlers.csv", index=False,  
              sep=";", encoding="utf-8-sig")  
print("brawlers.csv salvo!")
```

O gráfico da demo

E para fechar, o gráfico que vocês viram na segunda:

```
import matplotlib.pyplot as plt

grafico = top10.iloc[::-1] # o maior fica em cima
plt.barh(grafico["name"], grafico["trophies"])
plt.title("Top 10 brawlers por troféus")
plt.xlabel("Troféus")
plt.tight_layout()
plt.show()
```

Terminou? Desafios

1. Quantos brawlers seus têm menos de 1000 troféus? E qual é a média deles?
2. Qual brawler tem a maior sequência de vitórias (coluna `maxWinStreak`)?
3. Salve só o top 10 num CSV e abra no Excel
4. **Difícil:** com o `battlelog_exemplo.json` do repositório, descubra em qual modo de jogo o jogador mais ganha

Intervalo



Voltamos às 10h30

Desafio livre



Vocês escolhem a API

Como funciona

1. Formem duplas
2. Escolham uma API da lista do próximo slide
3. Definam **uma** pergunta que os dados podem responder
4. Escrevam um script que busca os dados e responde a pergunta
5. Às 11h30, algumas duplas mostram o resultado em 2 minutos

A lista: todas sem chave

API	Endereço para começar
PokeAPI	<code>pokeapi.co/api/v2/pokemon/pikachu</code>
Rick and Morty	<code>rickandmortyapi.com/api/character</code>
Harry Potter	<code>hp-api.onrender.com/api/characters</code>
Brasil API	<code>brasilapi.com.br/api/feriados/v1/2026</code>
Open-Meteo	<code>api.open-meteo.com/v1/forecast</code>
CoinGecko	<code>api.coingecko.com/api/v3/coins/markets</code>
Dog API	<code>dog.ceo/api/breeds/list/all</code>

Por onde começar

1. Abram a URL no navegador e olhem o JSON: onde está a lista?
2. Façam o `requests.get` e confirmem o status
3. Joguem a lista no `pd.DataFrame`
4. Respondam com filtro, `sort_values`, `value_counts` ou `groupby`
5. Travou? Chamem a gente

Exemplo: personagens por casa

Com a Harry Potter API, a pergunta inteira cabe em poucas linhas:

```
import pandas as pd
import requests

url = "https://hp-api.onrender.com/api/characters"
personagens = pd.DataFrame(requests.get(url).json())

com_casa = personagens[personagens["house"] != ""]
print(com_casa["house"].value_counts())
```

Ideias de perguntas

- **PokeAPI:** qual dos seus 5 pokémons favoritos é o mais pesado?
- **Rick and Morty:** quantos personagens estão vivos, mortos ou desconhecidos?
- **Brasil API:** quantos feriados de 2026 caem em dia de semana?
- **Open-Meteo:** qual vai ser o dia mais quente da semana em Prudente?
- **CoinGecko:** qual das 10 maiores criptomoedas mais subiu em 24 horas?

Mostrem o que descobriram

- 2 minutos por dupla
- Qual API vocês usaram?
- Qual era a pergunta?
- Qual foi a resposta? Mostrem o script rodando

Nesses dois dias, vocês aprenderam a

- Fazer requisições a APIs e ler o status da resposta
- Navegar em JSON como dicionário e se autenticar com chave
- Transformar JSON em tabela com o pandas
- Filtrar, ordenar, contar e agrupar para responder perguntas
- Ler a documentação de uma API nova e fazer a primeira requisição sozinhos

Para onde ir depois

- **POST** e APIs de escrita: mandar dados em vez de só buscar
- **OAuth:** o "entrar com o Google" que muitos apps usam
- **Visualização:** mais gráficos com o matplotlib
- **Coleta agendada:** rodar um script todo dia e montar o seu próprio histórico de dados

Obrigado!

Conte o que achou (1 minuto, anônimo):

tinyurl.com/secompp-feedback

Código e dados do minicurso:

tinyurl.com/secompp-apis



Antes de sair

Este computador é usado por todos os minicursos da semana.

- Saia de qualquer conta em que você entrou
- Feche o navegador